

LEGALEASE

Develop Core Functionalities

20-Page Professional Project Documentation

PROJECT DEVELOPMENT SCOPE

- 1. Brainstorming & Ideation** — identify the legal-problem space and define the product vision.
- 2. Requirement Analysis** — translate user needs into functional and non-functional requirements.
- 3. Project Design** — establish architecture, modules, data flow, interface, and security boundaries.
- 4. Project Planning** — organize milestones, resources, risks, dependencies, and deliverables.
- 5. Development** — implement the core LegalEase workflows and reusable service components.
- 6. Testing** — verify correctness, reliability, security, usability, and integration behavior.
- 7. Documentation** — record setup, APIs, modules, data structures, workflows, and maintenance guidance.
- 8. Demonstration** — present a realistic end-to-end user journey and expected outputs.
- 9. Diagrams** — communicate architecture, data flow, use cases, sequence, and module relationships.
- 10. Images / Visuals** — provide suitable wireframe-style visual references and project illustrations.

Integrated project documentation for academic and demonstration use.

Project: LegalEase

Focus: Development of core functionalities for a structured legal-information assistance platform.

Document: Detailed design, development, testing, documentation and demonstration report.

Prepared for: Academic / project submission and presentation.

Version: 1.0

1. Brainstorming and Ideation

LegalEase is conceived as a software platform that helps users organize and access legal information through a simple digital workflow. The ideation phase begins with a practical observation: legal processes can involve unfamiliar terminology, fragmented information, multiple documents, deadlines, and uncertainty about the next procedural step. The project therefore focuses on information organization, guided interaction, document support, and transparent explanations rather than presenting itself as a substitute for a qualified lawyer.

The central idea is to convert a complicated legal-information journey into a sequence of understandable tasks. A user can describe a general issue, identify a category, review relevant information, store supporting documents, track important actions, and generate a structured summary. The system can also provide educational explanations and references while clearly separating general information from professional legal advice.

Ideation objectives

- Reduce friction when users search for basic legal information.
- Present legal concepts using plain, structured language.
- Create a consistent workflow from issue identification to information review.
- Allow users to organize documents and case-related notes in one place.
- Provide reusable backend services so the platform can grow safely.
- Design a system that is testable, modular, auditable, and maintainable.
- Keep security and privacy as core design concerns from the beginning.
- Support future integration with authenticated users, databases, and external legal-information sources.

Problem-to-solution mapping

Observed problem	LegalEase response
Information is scattered	Centralized categories, search and structured results
Legal terminology is difficult	Plain-language explanations and glossary support
Users do not know what to do next	Guided workflow and checklist-style actions
Documents are difficult to organize	Document metadata and secure storage workflow
Important dates can be missed	Deadline/task tracking module
Users need a summary	Structured case-information summary generation
Systems become difficult to maintain	Layered architecture and modular services

The ideation outcome is a clearly bounded product concept: LegalEase should assist with legal-information organization and workflow management, while responsible use, privacy, security, and professional-review boundaries are visible throughout the product.

2. Requirement Analysis

Requirement analysis converts the project idea into measurable software behavior. For LegalEase, requirements are divided into functional requirements, non-functional requirements, data requirements, security requirements, and usability requirements. Each requirement should be testable so that development progress can be evaluated objectively.

Functional requirements

ID	Requirement	Expected behavior
FR-01	User access	The system can support registration/login when authentication is enabled.
FR-02	Issue intake	User can enter a structured description of a legal-information need.
FR-03	Category selection	System can classify an issue into configurable categories.
FR-04	Information search	User can search and filter available legal-information content.
FR-05	Document management	User can add document metadata and associate documents with an issue.
FR-06	Task/deadline tracking	User can create tasks and dates related to an issue.
FR-07	Summary	System can generate a structured summary from stored information.
FR-08	History	System can retain relevant activity records for authorized users.
FR-09	Validation	Input data is validated before processing or storage.
FR-10	Error handling	System returns consistent errors without exposing sensitive internals.

Non-functional requirements

- Performance: common API operations should respond promptly under expected project load.
- Reliability: failures should return controlled responses and should not corrupt stored records.
- Security: authentication, authorization, input validation, secure secrets, and safe error handling must be considered.
- Privacy: personal and legal-related information should be collected only when needed and handled carefully.
- Maintainability: modules should have clear responsibilities and predictable interfaces.
- Scalability: service boundaries should permit additional features without rewriting the whole system.
- Usability: screens and messages should be understandable to users with limited technical knowledge.
- Testability: business logic should be separable from framework and infrastructure concerns.

A requirements traceability approach should connect every important requirement to a module, test case, and demonstration step. This prevents features from being documented without actually being implemented or tested.

3. Project Design — System Architecture

The LegalEase design uses a layered architecture. The presentation layer handles user interaction, the API layer exposes controlled endpoints, the service layer implements business rules, the data-access layer communicates with persistent storage, and supporting services handle authentication, file processing, validation, logging, and configuration. This separation reduces coupling and makes the project easier to test.

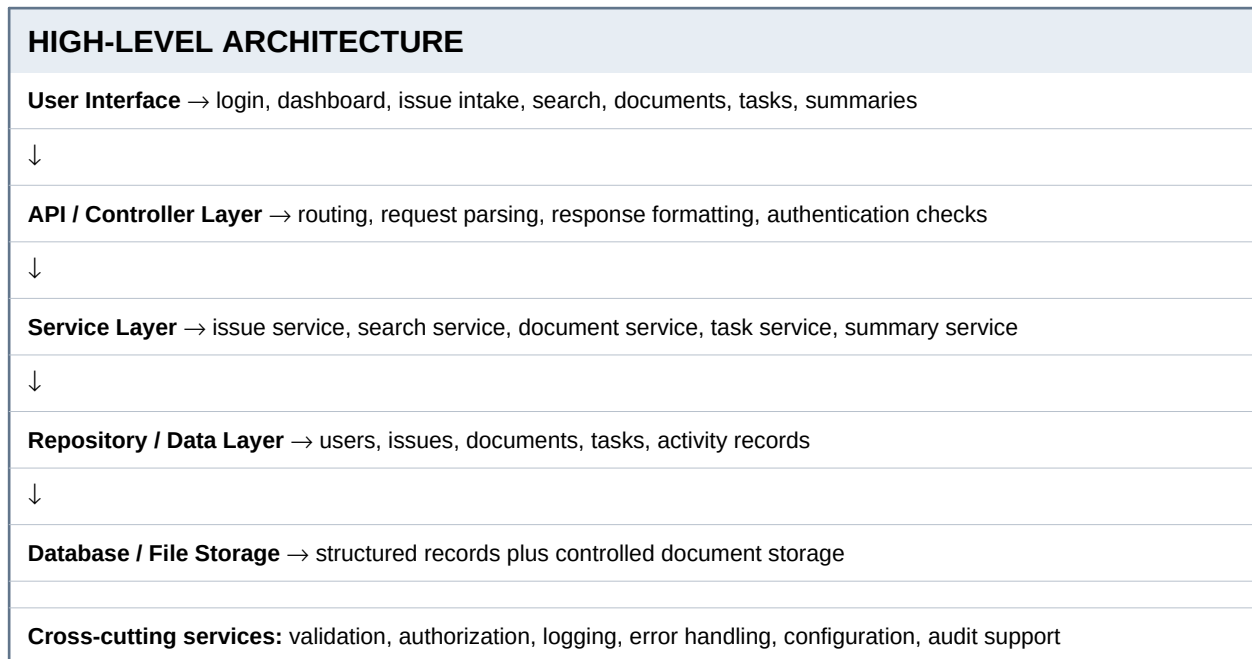


Figure 1. Conceptual architecture of LegalEase.

Core modules

Module	Purpose
Authentication	Identity, session/token validation, and access control.
Issue Management	Create, update, classify, and retrieve legal-information issues.
Knowledge/Search	Search and present approved legal-information content.
Document Management	Track documents, metadata, ownership, and permitted operations.
Task Management	Track actions, priorities, due dates, and status.
Summary Service	Convert stored issue data into a structured human-readable summary.
Audit/Logging	Record important system events without exposing sensitive data in logs.
Administration	Manage categories, content, permissions, and system configuration.

The design intentionally avoids placing business rules directly inside route handlers. This allows the same business logic to be tested independently and reused by future interfaces.

4. Project Planning

Project planning converts the architecture into a sequence of manageable development activities. A practical plan starts with the smallest useful end-to-end workflow and then expands it. Each milestone should produce a visible artifact: a design decision, working API, test result, diagram, or demonstration screen.

Phase	Major activities	Deliverable
1. Initiation	Problem definition, scope, assumptions	Project scope document
2. Analysis	Requirements, user stories, acceptance criteria	Requirements specification
3. Design	Architecture, database model, API design	Design specification
4. Foundation	Project structure, configuration, error handling	Runnable application skeleton
5. Core development	Issue, search, documents, tasks, summaries	Working core APIs
6. Security	Auth, authorization, validation, safe logging	Security controls
7. Testing	Unit, integration, API and usability tests	Test report
8. Documentation	Installation, API reference, workflows	User/developer documentation
9. Demonstration	Scenario, screenshots, expected outputs	Demo script and evidence
10. Finalization	Review, corrections, packaging	Final project build/report

Risk register

Risk	Impact	Mitigation
Incorrect legal information	High	Use controlled sources, timestamps, review workflow, and clear informational disclaimers.
Unauthorized access	High	Authentication, role checks, least privilege, secure session/token handling.
Sensitive data exposure	High	Minimize collection, encrypt where appropriate, restrict logs, validate uploads.
Scope expansion	Medium	Freeze MVP scope and use a prioritized backlog.
Poor test coverage	Medium	Map requirements to test cases and automate repeatable tests.
Dependency failure	Medium	Pin compatible versions and document setup requirements.
Unclear user workflow	Medium	Use prototypes, task-based testing, and simple navigation.

Planning should also define ownership, review checkpoints, version-control conventions, coding standards, backup strategy, and a change-management process.

5. Development — Project Setup and Configuration

Development begins with a clean project structure. A Python-based implementation can use a web framework such as FastAPI for API development, a validation library for request models, an ORM or repository pattern for database access, and a test framework such as pytest. The exact stack can be adjusted to project requirements, but the architectural boundaries should remain stable.

Suggested project structure

```
LegalEase/  
├── app/  
│   ├── main.py  
│   ├── core/  
│   │   ├── config.py  
│   │   ├── security.py  
│   │   ├── logging.py  
│   │   └── api/  
│   │       ├── routes_auth.py  
│   │       ├── routes_issues.py  
│   │       ├── routes_documents.py  
│   │       ├── routes_tasks.py  
│   │       └── routes_search.py  
│   ├── models/  
│   ├── schemas/  
│   ├── services/  
│   ├── repositories/  
│   └── utils/  
├── tests/  
├── requirements.txt  
├── .env.example  
└── README.md
```

Application entry point — illustrative

```
from fastapi import FastAPI  
from app.api.routes_issues import router as issues_router  
from app.api.routes_tasks import router as tasks_router  
  
app = FastAPI(title="LegalEase API", version="1.0.0")  
  
app.include_router(issues_router, prefix="/api/issues", tags=["Issues"])  
app.include_router(tasks_router, prefix="/api/tasks", tags=["Tasks"])  
  
@app.get("/health")  
def health():  
    return {"status": "ok", "service": "LegalEase"}
```

The sample is intentionally small: the main application should assemble routers and infrastructure rather than contain the full business logic. Environment-specific values such as database URLs and secret keys should be loaded from secure configuration rather than hard-coded.

Development standards

- Use version control with meaningful commits.
- Keep secrets outside source code.
- Validate all externally supplied input.
- Return consistent API response structures.
- Use type hints and descriptive names.
- Document important architectural decisions.
- Write tests alongside new business functionality.

6. Development — Core Issue Management

Issue management is the central workflow of LegalEase. An issue represents a user's legal-information request or a structured matter that the user wants to organize. The module should store only information necessary for the defined workflow and should enforce authorization so that one user's records are not exposed to another user.

Issue data model

Field	Type	Purpose
id	UUID / integer	Unique issue identifier
owner_id	UUID / integer	Associates the issue with its authorized user
title	String	Short issue name
category	String / enum	Configurable issue category
description	Text	Structured user-provided description
status	Enum	Open, in-progress, resolved/closed as configured
priority	Enum	Optional workflow priority
created_at	DateTime	Creation timestamp
updated_at	DateTime	Last update timestamp
metadata	JSON	Optional non-sensitive structured attributes

Illustrative service logic

```
def create_issue(user_id, payload):
    validate_issue_payload(payload)
    issue = Issue(
        owner_id=user_id,
        title=payload.title.strip(),
        category=payload.category,
        description=payload.description.strip(),
        status="OPEN",
    )
    return issue_repository.save(issue)

def get_issue(user_id, issue_id):
    issue = issue_repository.get(issue_id)
    if not issue or issue.owner_id != user_id:
        raise NotFoundError("Issue not found")
    return issue
```

API examples

Method	Endpoint	Purpose
POST	/api/issues	Create an issue
GET	/api/issues	List authorized issues
GET	/api/issues/{id}	Retrieve one authorized issue
PATCH	/api/issues/{id}	Update permitted fields
DELETE	/api/issues/{id}	Remove or archive according to policy

The important design principle is ownership enforcement. A valid identifier alone must never be treated as proof that the requesting user is authorized to access the record.

7. Development — Search and Legal-Information Knowledge

The knowledge/search component provides structured access to approved legal-information content. It should not silently invent laws, citations, case outcomes, or deadlines. Content should have metadata such as jurisdiction, topic, source, publication/update date, and review status. Where information may change, the interface should show its source context and date.

Knowledge item model

Attribute	Description
title	Human-readable content title
topic	Legal-information category
jurisdiction	Region or legal system to which the information applies
summary	Plain-language overview
source	Reference to the source or authority
effective_date	Date the information became applicable, where known
reviewed_at	Date the content was last reviewed
status	Draft, reviewed, archived, etc.
keywords	Searchable terms
disclaimer	Context and limitation notice where appropriate

Search workflow

SEARCH / RETRIEVAL FLOW
1. User query → normalize text and validate parameters
2. Filters → category, jurisdiction, date, content status
3. Retrieval → database/full-text index returns candidate records
4. Ranking → relevance and approved-content status are considered
5. Presentation → concise result cards with source metadata
6. Detail view → full summary, source, date, and related information
7. User action → save, link to issue, or add a task

Figure 2. Knowledge search workflow.

A future enhancement could add semantic search, but the project should first establish a reliable baseline using deterministic filters and indexed text. This makes behavior easier to test and explain.

Responsible information boundary

LegalEase should clearly communicate that general information is not individualized legal advice. The product should encourage users to consult an appropriately qualified professional when the situation requires professional interpretation, representation, or jurisdiction-specific advice.

8. Development — Document Management

Legal documents may contain sensitive personal information. Therefore, document management requires stricter controls than ordinary text fields. The design should separate document metadata from the binary file, restrict access by ownership or role, validate file type and size, and use safe storage names rather than trusting user-supplied filenames.

Document workflow

DOCUMENT PROCESSING PIPELINE
User selects file → client validates basic constraints → secure upload endpoint
→ server validates authenticated user → file type/size validation
→ malware/security scanning where available → safe storage
→ metadata record created → document linked to authorized issue
→ access controlled through authorization check → controlled download/view
→ audit event recorded without logging document contents

Figure 3. Controlled document lifecycle.

Document metadata

Field	Purpose
document_id	Unique identifier
owner_id	Authorization boundary
issue_id	Optional issue association
original_name	Display-only filename; never trusted for storage
stored_name	Generated safe storage identifier
mime_type	Validated content type
size	Enforced size limit
checksum	Optional integrity verification
uploaded_at	Upload timestamp
status	Active, quarantined, deleted/archived

Security controls

- Use authentication before accepting private document operations.
- Perform authorization checks on every read, update, and delete operation.
- Reject unexpected file types and enforce size limits.
- Do not construct storage paths directly from user input.
- Do not expose internal storage paths in API responses.
- Do not place document contents or secrets into application logs.
- Use encryption at rest and in transit where supported by the deployment environment.
- Define retention and deletion behavior before production deployment.

9. Development — Task, Deadline and Summary Functions

A legal-information platform becomes more useful when it converts information into actionable organization. LegalEase can provide a task module for user-defined actions such as collecting a document, reviewing a source, contacting an organization, or preparing questions for a professional consultation. The module should not autonomously create legal obligations; it should organize actions the user chooses to track.

Task model

Field	Example
task_id	Unique identifier
issue_id	Related issue
title	Review a document
description	Optional details
priority	Low / Medium / High
due_at	User-selected date/time
status	Pending / In Progress / Completed
created_at	Creation timestamp
completed_at	Completion timestamp

Summary generation

The summary service combines approved fields from an issue, associated tasks, selected documents, and user notes into a structured view. A summary should preserve source information and should never imply that generated text is a legal conclusion. The output can include: issue title, category, user-provided description, relevant documents, open tasks, important dates, and information sources.

```
summary = {  
  "issue": issue.title,  
  "category": issue.category,  
  "description": issue.description,  
  "documents": document_service.list_for_issue(issue.id),  
  "open_tasks": task_service.list_open(issue.id),  
  "sources": knowledge_service.related_sources(issue.category),  
}
```

Example user journey

- User creates an issue with a short title and description.
- System validates the input and stores the issue.
- User adds relevant documents and notes.
- User searches approved information by category and jurisdiction.
- User saves useful sources to the issue.
- User creates tasks and optional due dates.
- Dashboard shows open issues, pending tasks, and saved sources.
- User opens the summary to review the complete organized record.

10. API Design and Validation

A consistent API contract allows the frontend and backend to evolve independently. Every endpoint should define request fields, response fields, status codes, validation rules, authentication requirements, and failure behavior. API documentation should be generated from the application schema where possible, then reviewed for accuracy.

Example request schema

```
from pydantic import BaseModel, Field

class IssueCreate(BaseModel):
    title: str = Field(min_length=3, max_length=120)
    category: str = Field(min_length=2, max_length=60)
    description: str = Field(min_length=10, max_length=5000)
```

Validation principles

- Reject missing required values with a clear client-safe message.
- Enforce maximum lengths to reduce abuse and accidental oversized payloads.
- Normalize fields where appropriate without destroying meaningful user content.
- Use allow-lists for controlled categories and status values.
- Validate identifiers and dates before database queries.
- Never rely on frontend validation alone.
- Return machine-readable error structures for client applications.
- Do not reveal SQL statements, stack traces, secret values, or internal filesystem paths.

Suggested response structure

```
{
  "success": true,
  "data": {
    "id": "ISSUE-ID",
    "title": "Example issue",
    "status": "OPEN"
  },
  "message": "Issue created successfully"
}
```

Error structure

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid"
  }
}
```

Stable error codes make frontend behavior predictable. Human-readable messages can change without forcing clients to parse free-form text. For production, API versioning and backward compatibility should be considered before public release.

11. Database and Data Flow Design

The database design should represent the relationships among users, issues, knowledge content, documents, tasks, and audit records. The exact database technology can vary, but relational storage is suitable for structured ownership and relationships. Indexes should be added to fields frequently used for authorization, filtering, and retrieval.

CONCEPTUAL DATA RELATIONSHIPS
USER 1 ■■■■ * ISSUE
ISSUE 1 ■■■■ * TASK
ISSUE 1 ■■■■ * DOCUMENT
ISSUE * ■■■■ * KNOWLEDGE_ITEM through saved-source association
USER 1 ■■■■ * AUDIT_EVENT
ADMIN → manages approved KNOWLEDGE_ITEM records

Figure 4. Conceptual entity relationship diagram.

Data flow for issue creation

Step	Operation	Control
1	Receive POST request	Authentication
2	Validate request body	Schema validation
3	Resolve current user	Identity context
4	Construct service object	Business rules
5	Persist issue	Database transaction
6	Record event	Controlled audit log
7	Return response	Safe serialization

Database quality controls

- Use primary keys and foreign keys to preserve relationships.
- Use constraints for required values and controlled statuses.
- Use transactions for multi-record operations.
- Index owner_id and issue_id for authorization and retrieval.
- Store timestamps consistently and document timezone assumptions.
- Back up production data according to an approved retention policy.
- Test migrations on a copy before production deployment.
- Prevent direct database access from user-facing clients.

A well-designed data layer makes the application easier to evolve. It also gives testing a clear target because repository behavior can be tested separately from service rules and API routing.

12. Security and Privacy Design

Security is a continuous engineering responsibility. LegalEase may process personal narratives, documents, dates, contact information, and other sensitive records. The project therefore needs a threat-aware design even when it is developed as an academic prototype.

Security areas

Area	Implementation focus
Authentication	Strong identity verification and secure session/token management.
Authorization	Ownership and role checks on every protected resource.
Input security	Schema validation, length limits, allow-lists, safe parsing.
Secrets	Environment/secret manager; never commit credentials.
Transport	HTTPS/TLS in deployed environments.
Storage	Controlled database and file permissions; encryption where appropriate.
Logging	Useful operational events without sensitive content.
Uploads	File validation, size limits, safe names, scanning where available.
Dependencies	Regular review and controlled versions.
Recovery	Backups, restore tests, incident procedures.

Threat-to-control examples

- Broken access control → verify the authenticated owner or authorized role before every resource operation.
- Injection → parameterized queries/ORM operations and strict validation.
- Credential exposure → secure secret storage and rotation.
- Malicious file upload → allow-list types, size limits, safe storage and scanning.
- Information leakage → generic external errors and restricted logs.
- Session misuse → expiration, secure transport, and appropriate token handling.
- Data loss → backups and tested restoration procedures.

Privacy-by-design principles

Collect only data needed for the declared feature. Explain why information is collected. Provide appropriate deletion or retention behavior. Restrict internal access. Avoid using real sensitive legal documents in demonstrations unless the user has explicit permission and the data is appropriately protected or anonymized.

13. Testing Strategy

Testing verifies that LegalEase meets its requirements and behaves safely when users provide valid, invalid, incomplete, or unexpected inputs. Testing should begin during development rather than being postponed until the final week.

Testing levels

Level	Scope	Example
Unit	Individual functions/services	Issue validation accepts valid data and rejects invalid data.
Integration	Multiple modules	API → service → repository creates a record correctly.
API	HTTP contract	POST /api/issues returns expected status and schema.
Security	Access boundaries	User A cannot read User B's issue.
UI/Usability	User workflow	A user can find an issue and create a task without confusion.
Regression	Existing behavior	New document feature does not break issue creation.
Performance	Response and throughput	Common operations remain responsive under expected load.

Sample test cases

TC	Scenario	Expected result
TC-01	Create valid issue	201/200-style success response and stored record
TC-02	Missing title	Validation error
TC-03	Oversized description	Validation error
TC-04	Get own issue	Record returned
TC-05	Get another user's issue	Access denied or not-found behavior
TC-06	Upload unsupported file	Upload rejected
TC-07	Create task for valid issue	Task stored and linked
TC-08	Complete task	Status and completion timestamp updated
TC-09	Search by category	Only matching approved results returned
TC-10	Server failure	Controlled error without sensitive internals

Testing workflow

A practical cycle is: define acceptance criteria → write test → implement feature → run unit tests → run integration/API tests → inspect logs → correct defects → rerun regression suite → record evidence. Test results should include date, environment, test version, pass/fail status, and defect references.

14. Documentation Plan

Professional documentation allows another developer or evaluator to understand the system without depending on informal explanations. LegalEase documentation should be maintained alongside development and updated whenever an interface, workflow, data model, or security control changes.

Required documentation set

- Project overview and objectives.
- Scope, assumptions, constraints, and out-of-scope items.
- Functional and non-functional requirements.
- Architecture and module descriptions.
- Database/entity relationship documentation.
- API endpoint reference and request/response examples.
- Installation and environment setup guide.
- Configuration and environment-variable reference.
- Security and privacy guidance.
- Testing strategy, test cases, and results.
- Demonstration workflow and expected outputs.
- Troubleshooting and common errors.
- Maintenance and future enhancement plan.
- Version history and change log.

README structure

```
# LegalEase
## Overview
## Features
## Architecture
## Requirements
## Installation
## Environment Configuration
## Database Setup
## Running the API
## API Documentation
## Testing
## Security Notes
## Demonstration
## Troubleshooting
## Future Enhancements
```

Documentation quality checklist

Check	Question
Accuracy	Does the document match the current implementation?
Completeness	Are setup, features, testing and limitations included?
Clarity	Can a new developer follow the instructions?
Traceability	Can requirements be linked to tests and modules?
Security	Are secrets excluded and security assumptions stated?
Maintenance	Is the version/date visible and change history recorded?

15. Demonstration — End-to-End Workflow

The demonstration should show the project as a coherent system rather than as isolated screens. A short, repeatable scenario is preferable to a long sequence of unrelated features. The following scenario demonstrates the core functionality while using fictional, non-sensitive information.

Demo scenario

- Step 1: Open the LegalEase application and display the dashboard.
- Step 2: Sign in using a demonstration account.
- Step 3: Create a fictional issue such as a general consumer-information query.
- Step 4: Enter category, description, and optional priority.
- Step 5: Save the issue and show the generated identifier.
- Step 6: Search the approved knowledge area using a category and keyword.
- Step 7: Open a result and display its source metadata and review date.
- Step 8: Save the selected source to the issue.
- Step 9: Upload a fictional sample document or add document metadata.
- Step 10: Create two tasks related to the issue.
- Step 11: Assign one task a due date and mark another as completed.
- Step 12: Open the issue summary.
- Step 13: Show the combined issue, documents, tasks, and sources.
- Step 14: Display API documentation or an API request/response example.
- Step 15: Run the test suite and show the pass/fail summary.
- Step 16: Finish with security, privacy, limitations, and future-enhancement notes.

Expected demonstration outcome

At the end of the demonstration, the evaluator should be able to see a complete path from user input to validated storage, information retrieval, document/task organization, summary generation, and test evidence. The presenter should explicitly state that the demonstration uses fictional information and that the platform is intended for legal-information assistance rather than replacing professional legal advice.

Demonstration evidence

- Dashboard screenshot
- Issue creation screenshot
- Search results screenshot
- Document/task screen
- Summary screen
- API documentation screen
- Test results screenshot
- Architecture diagram

16. Diagrams and Visual Design

Diagrams make the architecture easier to understand during evaluation and presentation. LegalEase should include a consistent set of diagrams covering users, processes, data, components, and request flow.

Use-case view

USE-CASE DIAGRAM — TEXTUAL REPRESENTATION
User → Register/Login
User → Create / View / Update Issue
User → Search Legal Information
User → Save Sources
User → Add / View Documents
User → Create / Complete Tasks
User → View Summary
Administrator → Manage Categories and Approved Knowledge
Administrator → Review Content and Monitor System Events

Figure 5. Use-case representation.

Sequence view

ISSUE CREATION SEQUENCE
User → API: POST issue data
API → Auth: validate identity
API → Validator: validate payload
API → Issue Service: create_issue(user, payload)
Issue Service → Repository: save(issue)
Repository → Database: INSERT
Database → Repository: success
Repository → Service: issue record
Service → API: result
API → User: success response

Figure 6. Simplified sequence diagram.

Visual design guidance

Use a clean interface with consistent spacing, clear headings, readable status labels, and restrained use of color. Sensitive data should not be unnecessarily displayed in dashboard cards. Empty states should explain what the user can do next. Error messages should identify the corrective action without exposing implementation details.

17. Images / Suitable Project Visuals

For the final report and presentation, images should be purposeful. Instead of decorative screenshots that do not explain functionality, each visual should prove a requirement, demonstrate a workflow, or communicate architecture. The project can use screenshots of the actual prototype, wireframe-style interface images, diagrams, and test evidence.

Recommended image set

Image	Purpose	Suggested caption
Image 1	Login / access screen	LegalEase authentication interface
Image 2	Dashboard	LegalEase issue and task dashboard
Image 3	Issue form	Issue creation and validation interface
Image 4	Search	Legal-information search results with source metadata
Image 5	Document area	Controlled document metadata and upload workflow
Image 6	Task tracker	Tasks and due-date organization
Image 7	Summary	Structured issue summary
Image 8	API docs	Interactive API documentation
Image 9	Testing	Automated test execution evidence
Image 10	Architecture	LegalEase system architecture diagram

Image-generation / mockup prompts

Dashboard prompt: Create a clean professional legal-information assistance dashboard for a software project called LegalEase, showing fictional issues, task statuses, saved sources, and a simple navigation sidebar; modern academic software UI, readable typography, no real personal data.

Architecture prompt: Create a professional software architecture diagram for LegalEase showing User Interface, API layer, Service layer, Repository layer, Database, File Storage, Authentication, Validation, Logging, and Audit components with directional arrows and clear labels.

Search prompt: Create a realistic but fictional LegalEase search-results interface showing legal-information topics, jurisdiction filters, source labels, review dates, and a plain-language summary panel; no real case numbers or personal data.

Screenshot standards

- Use consistent viewport dimensions.
- Hide real credentials and personal information.
- Add figure numbers and captions.
- Keep text readable at report size.
- Use the same application name and terminology throughout.
- Capture both successful and validation/error states when useful.

18. Integration, Deployment and Maintenance

After the core modules work independently, integration connects them into one reliable application. Integration should be staged: first API and service integration, then database and storage integration, then authentication, then the complete user interface. Each stage should be tested before adding another dependency.

Deployment flow

DEPLOYMENT PIPELINE
Developer commit → automated checks → unit tests → build/package
→ integration/API tests → security checks → review/approval
→ staging deployment → acceptance testing → production deployment
→ monitoring/logging → backup and recovery controls

Figure 7. Recommended deployment lifecycle.

Environment configuration

Setting	Example purpose
APP_ENV	development / staging / production
DATABASE_URL	Database connection configuration
SECRET_KEY	Application secret managed outside source control
TOKEN_EXPIRY	Authentication/session policy
UPLOAD_LIMIT_MB	Maximum accepted upload size
LOG_LEVEL	Operational logging level
STORAGE_PATH	Controlled file-storage location
CORS_ORIGINS	Allowed frontend origins

Maintenance activities

- Review dependencies and security advisories.
- Back up databases and test restoration.
- Review error logs and operational metrics.
- Rotate secrets according to organizational policy.
- Review knowledge content for outdated information.
- Run regression tests before releases.
- Update API and user documentation when behavior changes.
- Review access permissions and inactive accounts.
- Track defects and enhancement requests in a backlog.

Future enhancements

Potential future work includes multilingual interfaces, richer document extraction, semantic search, configurable jurisdiction libraries, calendar integration, stronger audit dashboards, accessibility improvements, analytics, and carefully governed AI-assisted information retrieval. Such features should be added only after the core workflow is stable and their privacy, accuracy, and security implications are assessed.

19. Final Summary, Results and Conclusion

LegalEase: Develop Core Functionalities establishes a structured foundation for a legal-information assistance platform. The project begins with brainstorming and ideation, translates the problem into measurable requirements, designs a layered architecture, plans milestones and risks, implements core issue/search/document/task/summary workflows, and verifies behavior through a multi-level testing strategy.

Final deliverables

Area	Final artifact
Ideation	Problem statement, product vision, feature scope
Requirements	Functional and non-functional requirements
Design	Architecture, data model, module boundaries, workflows
Planning	Milestones, risks, quality checkpoints
Development	Core API/services and project structure
Testing	Test cases, automation plan, security checks
Documentation	README, API reference, setup and maintenance guide
Demonstration	End-to-end fictional user scenario
Diagrams	Architecture, data flow, sequence and use-case views
Images	Screenshots, mockups, visual evidence and captions

Project outcome

- LegalEase has a clear and modular technical direction.
- Core functions are separated into maintainable services.
- Requirements can be traced to implementation and tests.
- Security and privacy are incorporated into the design rather than treated as an afterthought.
- Documentation supports both academic evaluation and future development.
- Diagrams provide a common language for developers, evaluators, and users.
- Demonstration steps show how the major functions work together.
- Future enhancements can be added without discarding the core architecture.

Conclusion

The development of LegalEase core functionalities demonstrates how a software project can move systematically from an initial idea to a documented, testable, and demonstrable solution. The strongest implementation path is to keep the first release focused, validate each module, protect user data, document every important decision, and make the system's limitations visible. With these foundations in place, LegalEase can evolve into a broader platform while retaining a clear architecture, responsible information boundaries, and maintainable development practices.

End of 20-page LegalEase documentation. This document is suitable as a project-report foundation and can be extended with actual application screenshots, source-specific legal references, institution details, team information, and measured test results when those project artifacts are available.